

Quiz 6

Filename: q6.c

Instructions

Use the provided starter code, which you can access [here](#), and copy it in its entirety. This file is intentionally incomplete to simulate a written exam. You are strongly encouraged to first work out your solutions using pencil and paper. Once you are confident in your answers, type the final solutions into this file. You may only write code inside the bodies of the required functions. Do not modify function signatures, `#include` directives, or any other part of the file. Write your answers using a plain text editor only. Do not use a compiler, debugger, or IDE while completing this file. Save your file as `q6.c`. You may submit multiple times on Gradescope before the submission deadline. Only the final submission made before the deadline will be graded.

Important: Certain compiler warnings are treated as errors, and any such warning will cause compilation to fail. If your code does not compile on Gradescope, your submission will receive a score of 0. All successfully compiled submissions will be evaluated for both correctness and proper implementation.

Exam Preparation Tip

As you work through the problems, you must time yourself. Each question is designed to be completed within a reasonable amount of time. Some questions may take approximately 1 minute, others around 5 minutes, but no question should take more than 10 minutes.

For the following problems, use the following definitions of a binary tree node and a binary search tree node.

```
typedef struct BTreeNode_s {
    int data;
    struct BTreeNode_s *left;
    struct BTreeNode_s *right;
} BTreeNode;
```

TODO 1 (25 points)

A binary tree is considered balanced if, for every node in the tree, the absolute difference between the heights of its left and right subtrees is at most 1. Complete the function that returns 1 if the tree rooted at `r` is balanced. Otherwise, return 0.

Hint: An empty tree is considered balanced. Additionally, this problem is more naturally solved recursively.

You may assume that the following helper function is already defined:

```
int height(BTNode *n)
```

This function returns the height of the tree rooted at `n`.

```
int isBalanced(BTNode *r) {
```

```
}
```

TODO 2 (25 points)

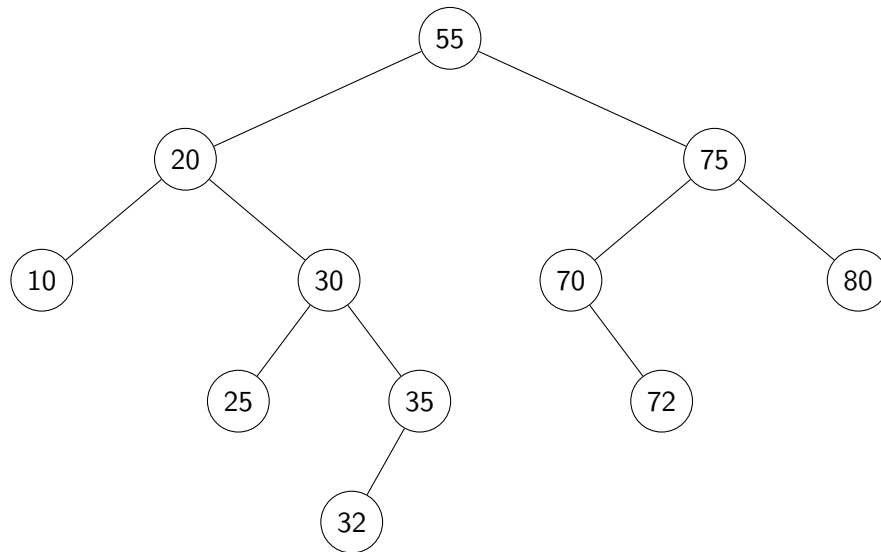
Complete the recursive function that prints the values of all nodes in the binary search tree that have exactly two children. The values must be printed in ascending order, with one value per line.

```
void twoChildren(BTNode *r) {
```

```
}
```

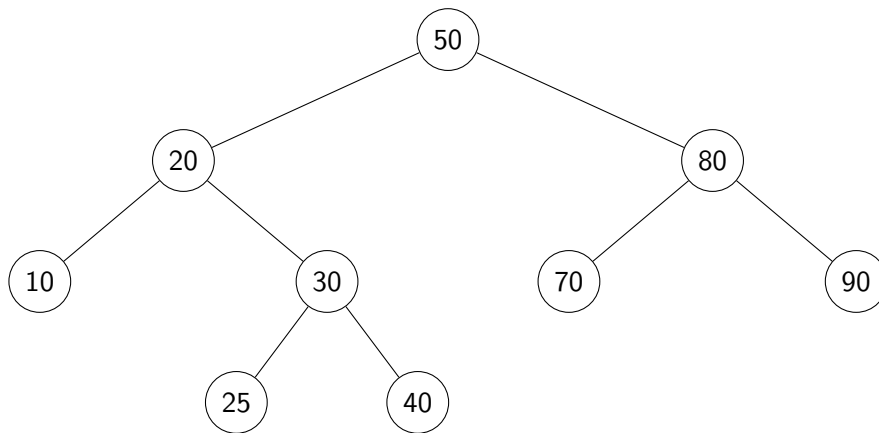
TODO 3 (15 points)

Consider the following binary search tree. Redraw the tree after deleting the node with the value **55**. Assume that deletion is performed using the delete-by-copying method. When necessary, replace the deleted node with its successor.



TODO 4 (20 points)

Given the following binary search tree, what value does the function `foo` return when called using the root of the tree shown below and `k = 30`?



```
int foo(struct BTreeNode *root, int k) {
    if( !root )
        return 0;

    int s = foo(root->left, k) + foo(root->right, k);

    if( root->data > k )
        s += root->data;

    return s;
}
```

TODO 5 (15 points)

Reconstruct and draw the binary search tree that produces the following traversal results:

- **Preorder:** 100, 75, 10, 25, 50, 125, 200, 175, 150
- **Inorder:** 10, 25, 50, 75, 100, 125, 150, 175, 200
- **Postorder:** 50, 25, 10, 75, 150, 175, 200, 125, 100